



CHALLENGE 09 / 20

THE POISONED BUILD SERVER

SOFTWARE SUPPLY-CHAIN PROVENANCE VERIFICATION

NATIONAL LEVEL HACKATHON • 2026

POWERED BY SYNTHOQUEST PVT LTD

Student Project Development Challenge • No organizer-provided evidence package.

1 | PROBLEM BACKGROUND

A software vendor supplying payments middleware to 1,100 customers, including banks and government bodies, shipped version 7.4.2. A customer's reproducibility check found a native library whose bytes cannot be derived from the stated source revision. Build logs, commit history, the software bill of materials and code signing all validate.

The pipeline uses Git, three internal package mirrors, two CI platforms, shared build caches, a signing HSM run by a separate team and a CDN, with sixty service builds a day. Builds are not bit-for-bit reproducible. Build agents are ephemeral and the logs live on the build servers. A transitive dependency changed nine days earlier. The last forty releases may be affected, and some customers rebuild locally.

The vendor must say which artefacts are clean, which are not, where the divergence entered and who is exposed.

2 | PROBLEM STATEMENT

Develop a **Build Provenance Verification System** that determines, for any delivered artefact, whether it is a faithful product of its declared source and dependencies, localises the pipeline stage or component where any divergence entered, and identifies every affected release and customer artefact.

The system must do this without trusting logs or metadata from components that may be compromised, and must find divergence that is dormant until a condition is met.

Hard constraints

- Logs, bill of materials and signatures may all be forged.
- Builds are not bit-for-bit reproducible, so naive comparison fails.
- The divergence may be tiny and conditionally active.
- Legitimate toolchain and dependency changes must be separated from malicious ones.
- Teams must use benign marker changes. No functional malicious code may be created or used.

No evidence package or dataset is provided. Teams decide what to build, what data they need, how to simulate the environment and how to validate the result. No tools or architecture are prescribed.

3 | MANDATORY FEATURES

Teams must develop a working software solution. The system must provide the following features. How each is achieved is for the team to decide.

01	Pipeline Trust Modelling	Ability to model which stages and records can be trusted.
02	Non-Reproducible Build Comparison	Ability to compare artefacts despite legitimate build variation.
03	Divergence Localisation	Ability to locate the stage where divergence entered.
04	Dependency Provenance Verification	Ability to verify provenance across ecosystems and transitive dependencies.
05	Forged-Metadata Resilience	Ability to remain sound when logs and SBOMs are forged.
06	Dormant Divergence Detection	Ability to find divergence that only activates under conditions.
07	Release and Customer Impact Scoping	Ability to identify every affected release and deployment.
08	Verifiable Provenance Evidence	Ability to produce evidence a third party can check.

The system must clearly distinguish between:

VERIFIED	DIVERGENT	UNVERIFIABLE	COMPROMISED STAGE	UNKNOWN
----------	-----------	--------------	-------------------	---------

A provenance claim that cannot be verified must not be presented as established.

Build, don't search. This is a development challenge. Teams must create the system and the controlled environment that proves it works. Submitting an analysis of a given dataset is not a solution.

4 | EXPECTED OUTPUT

Live demonstration. The team must show that its system can handle a complex controlled scenario created by the team. The demonstration must show:

01	Teams' own simulated multi-stage build pipeline
02	A planted benign divergence at a chosen stage
03	Verification of clean and diverged artefacts
04	Operation with forged logs and metadata
05	Detection of a conditionally active divergence
06	Localisation of the stage and affected releases
07	Evidence a customer can independently verify

The team must be able to explain why the system reached each major conclusion.

Validation standard (every team)

- Build a scenario generator with known ground truth and evaluate on at least ten distinct scenarios, including ones with deliberately misleading evidence.
- At the demonstration, judges change scenario parameters on the spot. The system must adapt without being rebuilt.
- Include an adversary variant that actively tries to defeat the system, and report how the system held up.
- Stated confidence must match measured accuracy. Overconfident or unexplained conclusions lose marks.
- No real systems, real personal data or functional malware may be used.

Submission. Each team must submit:

01	Working Prototype	A functional implementation of the proposed solution.
02	Source Code	Complete source code required to reproduce the project.
03	Technical Architecture	Architecture and component description.
04	Validation Environment	A controlled environment created by the team to demonstrate the solution.
05	Technical Documentation	Implementation, design and testing documentation.
06	Forensic Investigation Report	A professional report generated from the demonstrated investigation.
07	Demonstration	Live technical demonstration to the judging panel.
08	Limitations	Known limitations and situations where the system cannot establish a reliable conclusion.